Mini-projet - Comparaison des structures de données

Fernandez Mathys,

Defois Thomas

projet sur github:

https://github.com/MathysFernandez-labs/C-project

Auto Évaluation :	A | B | C
Fernandez Mathys	A
Defois Thomas	A

Introduction:

Son objectif principal est d'implémenter et de comparer expérimentalement les performances des algorithmes d'insertion, de suppression et de recherche. Cette comparaison s'applique à quatre structures de données fondamentales: les arbres binaires, les arbres AVL, les arbres bicolores (Rouges et Noirs) ainsi que les tables de hachage.

Afin d'éprouver l'efficacité temporelle de nos implémentations, l'étude expérimentale a été menée sur différents types de données (entiers et chaînes de caractères), en faisant varier le volume de données de 100 à 10 000 éléments par pas de 100. Au-delà de la stricte conception algorithmique et de la résolution des complexités théoriques, une attention toute particulière a été portée à la gestion de projet. Le développement a été versionné et organisé de manière collaborative sur la plateforme **GitHub**. Pour assurer une traçabilité des modifications et faciliter la relecture au sein de l'équipe, nous avons adopté la norme des **Conventional Commits**.

De plus, nous avons mis en place une intégration continue via des actions GitHub, incluant un workflow automatisé pour garantir un suivi des tickets et optimiser notre gestion des tâches. Nous avons également mit en place des Issues Template pour faciliter le développement. Enfin, l'ensemble de notre dépôt est protégé et distribué sous une **Licence**, attestant du respect des standards actuels de l'ingénierie logicielle et du développement open-source.

Fonctionnement des Structures:

Pour ce projet, toutes nos structures ont été codées de manière générique en C afin de pouvoir stocker aussi bien des nombres entiers que des chaînes de caractères comme il était demander dans le sujet.

- La Table de Hachage : Elle utilise une fonction de hachage pour transformer la donnée en un indice de tableau. Cela permet de ranger l'élément directement dans la bonne case, mais si plusieurs données tombent sur la même case (collision), elles s'accumulent dans une liste qu'il faut ensuite parcourir. Rem: la gestion du nombre de case max du tableau est importante, si trop faible cela revient à stocké dans une liste et l’avantage de la fonction de hachage disparaît. L’indice max retourné par la fonction doit dépendre du nombre de données max auquel on doit s’attendre.

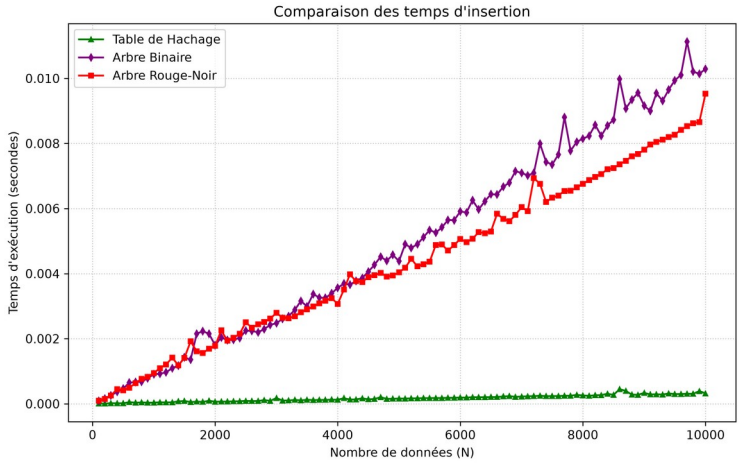
- L'Arbre Binaire de Recherche: Chaque élément possède au maximum deux enfants. Les valeurs plus petites vont à gauche et les valeurs plus grandes vont à droite. Sa rapidité dépend de l'ordre d'arrivée des données: s'il n'est pas équilibré, certaines branches deviennent très longues à parcourir.

- L'Arbre AVL : C'est un arbre binaire de recherche qui se rééquilibre automatiquement de manière très stricte après chaque ajout ou suppression. La différence de hauteur entre la branche gauche et la branche droite d'un nœud ne dépasse jamais de 1, ce qui garantit un arbre toujours parfaitement plat.

- L'Arbre (Bicolore) : Chaque nœud possède un pointeur vers son parent et une couleur (Rouge ou Noir). Lorsqu'on ajoute ou supprime une donnée, l'arbre applique des règles de couleurs et des rotations pour rester équilibré.

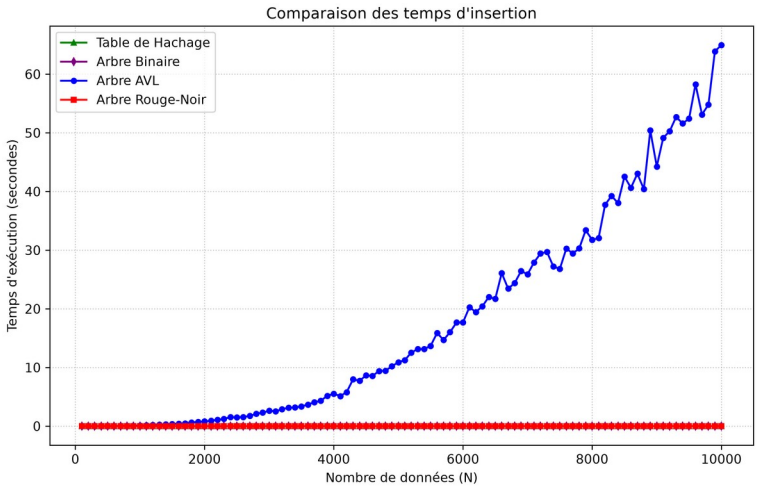
Analyse des Performances:

1) Temps d'insertion:

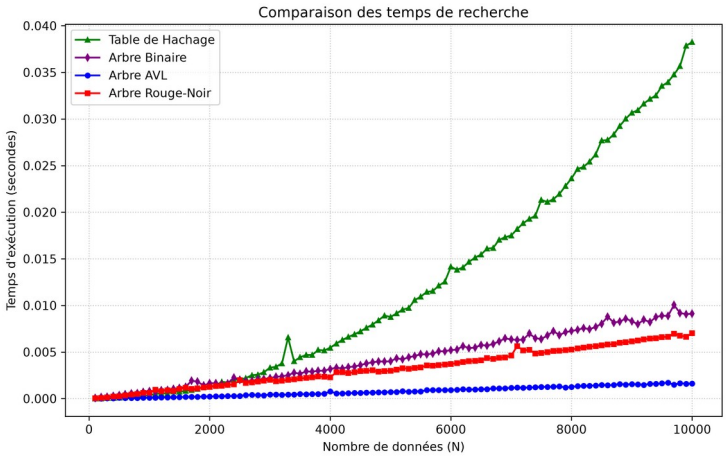


La courbe verte reste quasiment plate et collée à 0 seconde. Ajouter un élément est quasi instantané car il suffit de calculer son indice et de l'insérer directement en tête de liste sans chercher à trier les éléments. La complexité est 0(1), cependant on peut s'attendre à ce que cela ne soit pas le cas si la valeur max retourné par la fonction de hachage n'est pas assez grande, ce qui créerait des listes longues à chaque indice du tableau.

Cette version ajoute l'arbre AVL, on voit bien que les 3 autres structures sont écrasées par le pois de l'arbre AVL ce qui montre une grosse compléxité d'insertion, sûrement du au rééquilibrage générique utilisé et le calcul de la balance qui est récursif (utilisant height).

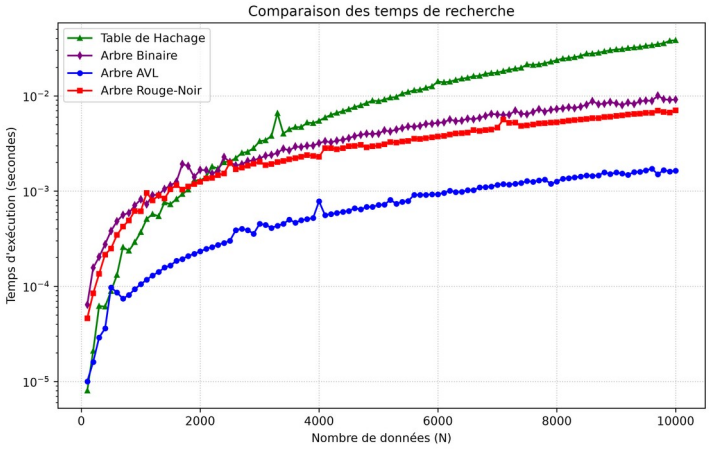


2) Temps de recherche:



L'Arbre AVL est le grand gagnant avec un temps inférieur à 0,002 seconde pour 10 000 recherches, car son équilibrage strict offre le chemin le plus court possible depuis la racine. L'Arbre Rouge-Noir arrive deuxième et bat l'Arbre Binaire simple. Le plus lent est la table de hashage qui est le seul avec une augmentation non constante et non nul de la complexité.

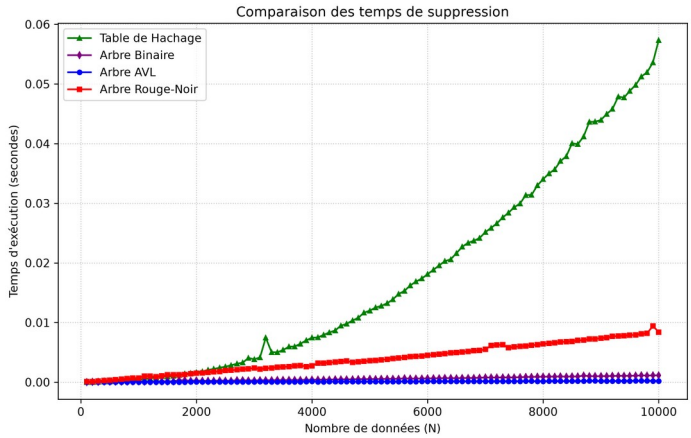
Lorsque le nombre de données grandit dans une table de taille fixe, les collisions se multiplient: la recherche finit par parcourir de longues listes chaînées de manière linéaire, ce qui dégrade fortement le temps total.



A l'aide de cette version logarithmique on peut mieux constater les différence temporelle lorsque les données sont faibles. On en conclu visuellement que la table de hachage est meilleur que l'arbre binaire et que l'arbre bicolore lorsque les données sont peu nombreuse.

3) Temps de suppression:

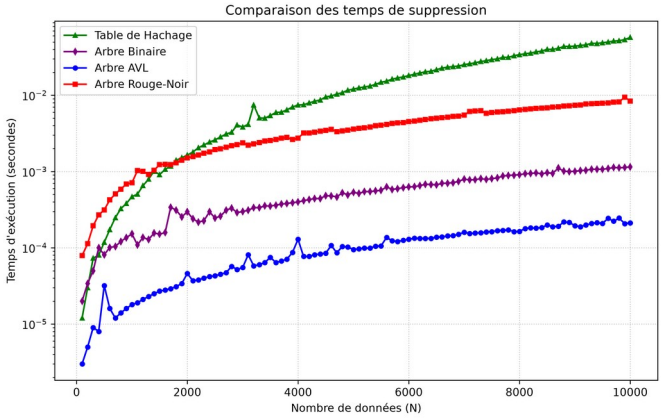
Le premier graphique montre l'écart global en secondes, tandis que le second utilise une échelle logarithmique pour mieux distinguer les courbes très proches de zéro..



Pour supprimer un élément, il faut d'abord le chercher, ce qui explique pourquoi la Table de Hachage reste la plus lente à cause du parcours des collisions.

Du côté des arbres, l'AVL et l'Arbre Binaire simple sont extrêmement rapides, tandis que l'Arbre Rouge-Noir monte progressivement en raison du coût de la fonction tree_delete_reequilibrage appelée après chaque retrait d'un nœud noir.

On peut voir grâce au graphique 2 (avec l’échelle logarithmique) que la table de hachage est temporairement meilleur que l’arbre rouge noir en fonction de la quantité de donnée (lorsque assez faible).



Bilan Comparatif:

Table de hachage

- Avantages : Offre les meilleures performances globales avec une complexité moyenne en O(1) pour l'insertion, la recherche et la suppression grâce à un accès direct par calcul d'indice. (tant que la valeur max retourner par la fonction de hachage est assez haute).

- Inconvénients : Ne conserve aucune relation d'ordre entre les clés et subit une dégradation vers O(n) si la fonction de hachage génère trop de collisions. (Rem : elle devient une liste de liste)

Arbre Binaire

- Avantages : Structure légère et simple à implémenter qui permet de conserver les données triées sans surcoût algorithmique lié à des rotations lors des modifications.

- Inconvénients : Dépend fortement de l'ordre d'arrivée des données ; l'insertion de valeurs déjà triées dégénère l'arbre en liste chaînée et fait chuter la complexité en O(n).

Arbre AVL

- Avantages : Son équilibrage strict garantit une hauteur minimale et offre les recherches les plus rapides parmi les structures arborescentes en O(log n).

- Inconvénients : Le maintien de cette condition stricte impose de fréquentes rotations lors des insertions et des suppressions, ce qui pénalise les performances en écriture.

Arbre Bicolore (Rouge et Noir)

- Avantages : Son équilibrage plus souple garantit une complexité en O(log n) tout en limitant le nombre de rotations surpassant l'AVL sur les modifications fréquentes.

- Inconvénients : Sa structure nécessite de stocker un pointeur vers le nœud père ainsi qu'un champ de couleur (Rouge ou Noire), et sa hauteur légèrement supérieure à l'AVL rend la recherche pure marginalement plus lente.

Conclusion du bilan, la **table de hachage** s'impose dès lors que la rapidité d'accès prime et qu'aucun tri n'est nécessaire. Si le maintien d'un ordre sur les clés est requis, l'**ABR simple** doit être écarté en production au profit d'arbres auto-équilibrés: l'**arbre AVL** sera privilégié pour des systèmes majoritairement sollicités en lecture, tandis que l'**arbre Rouge et Noir** constitue le meilleur compromis généraliste lorsque les insertions et suppressions sont régulières.